

FALCON: LLMs for Task-Driven Orchestration of Sensing Pipelines During Emergencies

Kevin Post
University of Tartu
Tartu, Estonia
kevin.post@ut.ee

Reo Kuchida
University of Tartu
Tartu, Estonia
reo.kuchida@ut.ee

Juan Gonzalo Quiroz Cadavid
University of Tartu
Tartu, Estonia
juan.gonzalo.quiroz.cadavid@ut.ee

Adeyinka Akintola
University of Tartu
Tartu, Estonia
adeyinka.jimoh.akintola@ut.ee

Abstract

Urban environments are increasingly embedded with dense sensing infrastructure, yet real-time access to relevant data remains limited, especially in privacy-sensitive, time-critical scenarios such as medical emergencies. Large Language Models (LLMs) offer a promising solution as they can generate executable logic from natural language prompts to coordinate heterogeneous devices. We introduce FALCON, an LLM-based system for real-time sensor data analysis in emergency response. We perform code generation for gesture recognition and QR code decoding using GPT-4o, Qwen2.5-coder, and Claude Code, benchmarking against a human-engineered baseline. FALCON achieves an F1 score of 0.801 in gesture detection, demonstrating competitive performance. We also explore decentralized deployment on Raspberry Pi devices. Our findings show FALCON can bridge fragmented sensing systems into scalable, adaptive, and privacy-aware urban intelligence pipelines.

CCS Concepts

• Human-centered computing → Ubiquitous and mobile computing.

Keywords

Large Language Model; Edge AI; Human Activity Recognition

ACM Reference Format:

Kevin Post, Reo Kuchida, Juan Gonzalo Quiroz Cadavid, and Adeyinka Akintola. 2025. FALCON: LLMs for Task-Driven Orchestration of Sensing Pipelines During Emergencies. In *Companion of the 2025 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp Companion '25)*, October 12–16, 2025, Espoo, Finland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3714394.3750598>

1 Introduction

Emergency and critical events, such as health-related crises or traffic accidents, often unfold in an instant, requiring specialised and sometimes decisive response strategies. In cases like cardiac

arrest or stroke, every second is critical: cardiopulmonary resuscitation (CPR) must be administered within 5 minutes, and certain stroke treatments are effective only within narrow therapeutic windows [8]. Traditionally, responders rely on their expertise and experience but could benefit from additional contextual information to guide treatment and decision-making. During emergencies, it is rarely feasible to conduct interviews or collect detailed input from witnesses. Fortunately, the growing presence of IoT devices, wearables, and personal sensors that continuously monitor activities offers a new source of real-time data. Sensors embedded in infrastructure, vehicles, or personal devices often record the moments before and during an emergency, making them valuable for both retrospective analysis and real-time support. However, extracting actionable insight from sensor data remains challenging. This is where LLMs emerge as a powerful tool to discover relevant devices, generate code to extract information, and synthesize insights from diverse inputs, enhancing emergency response capabilities.

Existing work has explored activity inference from multiple sensing sources in complex environments. During critical events, multiple sources often capture relevant data, such as smartphone recordings by bystanders, nearby surveillance cameras, and vehicle-mounted cameras [14]. These streams provide valuable insights and are typically used retrospectively, for example, in criminal investigations or insurance claims. Systems such as FollowMe [2] enable retrospective data retrieval, but are statically designed and require manual instrumentation, limiting adaptability in dynamic scenarios. LLMs have shown strong performance in interpreting sensor data [7] and generating code for sensor data collection [11]. However, it remains unclear whether LLMs can effectively support real-time decision-making by automating data collection and analysis from nearby devices.

We contribute FALCON, an LLM-based multi-agent system designed to support emergency response through real-time sensor data processing and analysis. The key idea is straightforward: in urgent situations, responders should be able to describe their needs in natural language, and the system should automatically retrieve and analyze the most relevant data. FALCON uses LLMs to orchestrate sensing pipelines in a task-driven manner, generating executable 'glue code' to fulfill specific emergency response needs. As illustrated in Figure 1, the process starts when a dispatcher issues a natural language query (Steps 1–2), the LLM interprets the request



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

UbiComp Companion '25, Espoo, Finland

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1477-1/2025/10

<https://doi.org/10.1145/3714394.3750598>

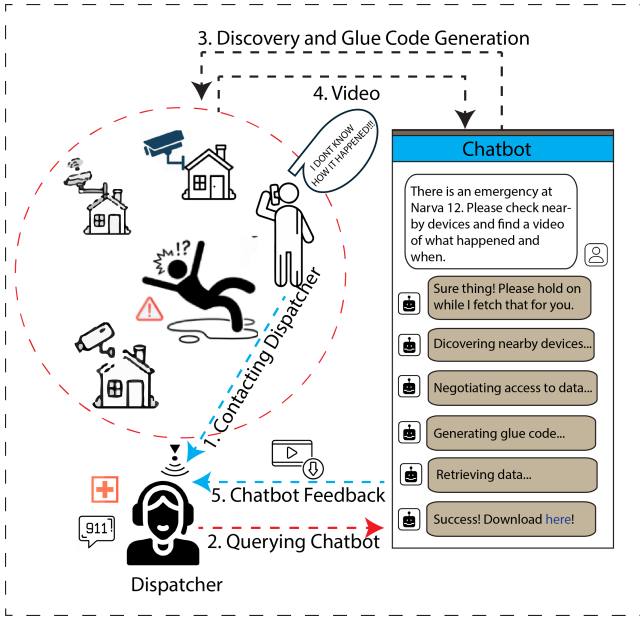


Figure 1: LLMs generate glue code to process sensor data from heterogeneous sources and provide information.

and generates executable glue code (Step 3), applies existing algorithms (Step 4), and returns meaningful data segments to support decision-making (Step 5). To test this approach, we built a prototype using off-the-shelf components including Raspberry Pi devices connected to a set of IP and USB cameras simulating real-world urban emergencies. We evaluated FALCON on two representative computer vision tasks, comparing the performance of LLMs (GPT-4o, Qwen2.5-coder) and LLM-based coding tool (Claude Code) to that of a human software engineer. Our results show that LLMs can generate functionally correct code, achieve an F1 score of 0.801 in detecting simple human-specific gestures, performing comparably to human-written solutions in both execution and accuracy. We also explore deploying lightweight LLM agents on edge devices to enable decentralized operation. These findings highlight the potential of FALCON to transform fragmented sensing infrastructure into an adaptive and privacy-aware urban intelligence system—capable of supporting emergency responses with minimal human setup.

2 Related Work

Data access in fragmented sensing environments: Traditional context-aware brokers and access control mechanisms (RBAC, ABAC, CapBAC) provide structured data sharing but rely on highly centralised infrastructure and introduce single points of failure, limiting scalability in dynamic environments [1, 6]. Blockchain-based approaches are a decentralized alternative, however they are still constrained by static policy logic and limited adaptability to runtime tasks [6]. LLMs offer adaptive brokering by interpreting natural language requests and generating task-specific logic dynamically [11].

Orchestration across heterogeneous devices: Real-time orchestration across sensing infrastructure requires determining which

devices to invoke, how to interface with them, and how to assemble their outputs to fulfill sensing objectives [9]. Traditional middleware systems [1] enable basic interoperability, but lack runtime flexibility. Recent work explores LLMs for decentralized device coordination [4] using multi-agent systems working adaptively toward natural language goals without hardcoded logic, showing the potential LLMs have for dynamically coordinating workflows, but focuses primarily on agent behavior [13] rather than practical sensing infrastructure interfacing.

Adaptive task execution: Beyond device discovery and access control, many applications require autonomous execution of task-specific logic, such as activity recognition or user alerts. Traditional solutions [2] rely on statically programmed modules, limiting their scalability. Recent work demonstrates that LLMs can generate executable "glue code" for robotics [11]. Our work extends this to runtime glue code generation for sensing applications with lightweight models on edge devices like Raspberry Pi.

3 Experiments

We tested whether LLMs can act as autonomous orchestrators by asking them to generate working code in two representative computer vision tasks. The first task was to detect a specific human gesture in video. The second task was to identify a QR code from video to show LLMs' adaptability to different task structures. These experiments help us understand the feasibility of our proposed approach. For comparison, we also asked two human software engineers to complete the same tasks.

3.1 Human Motion Detection

Procedure: Each system was asked to generate code to detect a predefined human gesture from video and identify when it occurred. We tested three LLMs. Claude Code is a command-line-based AI coding system that runs iteratively, executing and refining its own code based on feedback. We used the Sonnet 4 model in this tool. GPT-4o and Qwen2.5-coder 1.5B are prompt-response models; we interacted with them through text prompts and allowed re-prompting until they produced usable output. Two software engineers performed the same task independently, and we averaged their results. Each system ran five trials.

Task: The LLMs were tasked with generating Python code that identifies a specific motion in video: *a person clapping with their upper arms parallel to the ground and elbows flexed 90 degrees upwards*, imitating the clapping gesture found in the Opportunity++ dataset [5]. The code should return a list of time intervals (start and end in seconds) during which the motion occurs.

Dataset: A custom video dataset was created, including the target motion. Two scenarios were tested: (1) videos where only the target individual performs the motion, and (2) videos where the target appears alongside two distractor individuals, with only the target executing the motion. All videos were manually annotated with ground truth motion intervals. Figure 2a and 2b show a sample frame from the dataset.

Evaluation: Predicted motion intervals were compared with the ground truth using temporal overlap-based metrics: precision, recall, and F1 score.

3.2 QR Code Decoding for Identity Verification

Procedure: Each system was asked to generate code for detecting a QR code shown by the user. This experiment was performed only by the LLMs, with each system running ten trials. This setup shows how LLMs adapt to a different task structure by constructing new functional pipelines from user requests, without relying on fixed, pre-defined modules.

Task and Dataset: Each video shows a person briefly holding a phone that displays a QR code. The task for each system was to generate code that detects and decodes the QR content, which consists of a unique UUID string. Figure 2c shows a sample frame from the dataset.

Evaluation: A trial was counted as successful if the generated code ran without errors and the decoded string matched the ground truth UUID exactly. Each system was tested across ten trials, and we report the success rate as the percentage of correct decodings.

4 Results

4.1 Human Motion Detection

Table 1 reports LLM performance on interval detection under single- and multi-person conditions (see Figure 2). Claude Code consistently generated syntactically valid code and achieved high accuracy in single-person settings with minimal tuning. However, its performance dropped in multi-person scenes, likely due to difficulty isolating the target. GPT-4o performed similarly, though it occasionally produced incomplete or incorrect code on first attempt. This reflects known limitations in multi-step reasoning for vision tasks, though GPT-4o often recovered with minor prompt adjustments. Qwen2.5-coder failed to generate valid code across all trials, frequently hallucinating libraries and showing poor error recovery. Overall, results highlight that while advanced LLMs can handle simple perceptual tasks, they remain challenged by dynamic, multi-agent visual contexts without specialized prompting or architectural support.

To better understand the strengths and limitations of each system, we analyzed their behavior when handling errors during code generation and execution. Claude and GPT-4o both used MediaPipe and followed logic similar to the software engineer’s written reference code. However, their initial outputs failed due to poor parameter choices. Claude stood out by detecting that the output was empty, adjusting the parameters over multiple attempts, all triggered from a single initial prompt. In contrast, GPT-4o and Qwen2.5-coder required manual feedback to recover from errors. Claude’s ability to self-correct enabled a closed-loop workflow, showing promise for autonomous coordination in real-time or hands-free scenarios.

We also qualitatively analyzed the error behavior of the models during “glue code” generation. Qwen2.5-coder produced code with logic similar to that of GPT-4o and Claude Code, but was more prone to hallucinations. In three out of ten trials, the code failed due to non-existent attributes or incorrect usage of standard libraries such as PIL or OpenCV. Even when provided with error messages, Qwen2.5-coder often repeated the same mistakes, showing limited ability to correct itself based on feedback. Only one of the failed cases was resolved through a follow-up prompt. These results highlight Qwen2.5-coder’s weaker robustness in handling low-level API

Table 1: Performance of each system in detecting clapping intervals under two conditions: single-person vs. multi-person. Metrics are based on temporal overlap between predicted and ground truth intervals.

System	Recall	Precision	F1 Score
Single-Person Scenario			
Human Engineer (avg.)	0.981	0.901	0.939
Claude Code (Sonnet 4)	0.728	0.890	0.801
GPT-4o	0.105	1.000	0.191
Qwen2.5-coder 1.5B	Not Functional		
Multi-Person Scenario			
Human Engineer (avg.)	0.293	0.656	0.405
Claude Code (Sonnet 4)	0.518	0.563	0.539
GPT-4o	0.067	0.589	0.120
Qwen2.5-coder 1.5B	Not Functional		

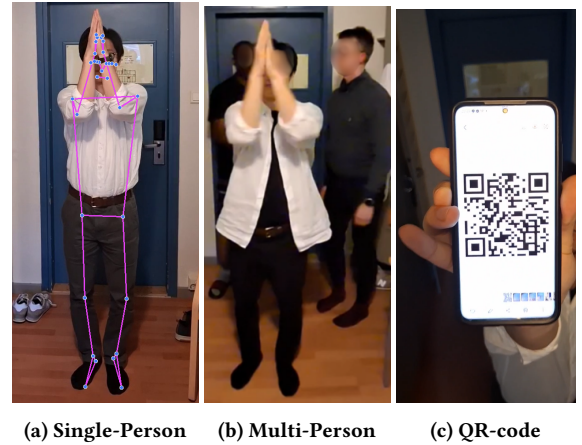


Figure 2: Experimental conditions: (a) single-person scenario (image taken from human engineered video), (b) multi-person scenario, (c) QR-code.

errors and a lack of effective self-correction compared to Claude Code and GPT-4o.

4.2 QR Code Decoding for Identity Verification

Claude Code and GPT-4o both achieved a 100% success rate, correctly decoding the UUID from the QR code in all ten trials. Qwen2.5-coder reached a 70% success rate. Compared to the gesture detection task, this simpler setup resulted in more stable and accurate code generation across all systems.

4.3 Inference Reliability

We evaluated the feasibility of using LLMs for code generation and execution by focusing on inference time and reliability. The goal was to understand the trade-offs between cloud-based and locally hosted models in real-time or near-real-time applications. Cloud-based LLMs such as Claude Code and GPT-4o demonstrated

significantly faster and more consistent inference times. Claude Code achieved a mean inference time of 7.07 ± 0.46 seconds, while GPT-4o followed with 9.24 ± 4.69 seconds. Both models maintained a 100% success rate, indicating high reliability across trials. These results suggest that cloud-based models are well-suited for scenarios requiring low-latency responses and stable performance. In contrast, the local model exhibited much longer inference times and higher variability. Qwen2.5-coder 1.5B was deployed and run directly on a Raspberry Pi 4 Model B with a quad-core ARM Cortex-A72 CPU at 1.5 GHz and 4 GB of RAM. It recorded a mean of 479.66 ± 87.42 seconds with a lower success rate of 70%. These results highlight the current limitations of running large-scale LLMs locally on constrained devices in terms of responsiveness and consistency.

Local models offer key advantages such as offline autonomy and data privacy. Moreover, running the LLM directly on a Raspberry Pi demonstrates the practicality of this approach. This shows that lightweight, low-cost hardware can support on-device code generation and execution, making it more accessible in environments where powerful edge servers are not available, such as public parks or other outdoor settings. While real-time interaction remains a challenge, local models are a viable solution for latency-tolerant tasks that benefit from fully local operation.

Table 2: Inference time (seconds) summary statistics for QR code decoding. Claude Code and GPT-4o are cloud-based, while Qwen2.5-coder 1.5B is running on Raspberry Pi.

Model	Mean $\pm$ SD	Min	Max
Claude Code (Sonnet 4)	7.07 $\pm$ 0.46	6.60	7.64
GPT-4o	9.24 $\pm$ 4.69	4.63	12.10
Qwen2.5-coder 1.5B	479.66 $\pm$ 87.42	392.25	623.32

5 Discussion

Ethical governance: Our system uses LLMs to negotiate data access contextually and on demand, supporting ethical governance by aligning human intent with enforceable policies. However, challenges arise due to the "black-box" nature of LLMs, which complicates auditing data access and decision-making. Additionally, slight changes in task descriptions can lead to unpredictable outcomes ("prompt drift"), affecting accountability [3].

Inclusivity in gesture-based authentication: While gesture-based authentication is minimally obstructive, it still favors able-bodied users [12]. However, LLMs allow gesture redefinitions for user-specific options. Yet, recognition performance depends on sensing quality, degrading in low-resolution or poor lighting, impacting under-resourced users. Thus, inclusivity could improve with multimodal fallbacks and adaptive prompts for accessibility.

Long-term sustainability in urban settings: LLMs facilitate scalable device adaptation via code generation, but rely on continuous updates from device APIs and libraries [4]. Often, APIs evolve faster than documentation, and proprietary formats remain underexposed, leading to a latency gap with new tech. Furthermore, outdated knowledge and hallucinations are common issues, as seen in our experiments. Integrating lightweight schemas into devices for real-time LLM consultation could mitigate these challenges.

Incorporating stakeholder feedback: Dynamic LLM-generated workflows allow customization but may lack reliability for high-stakes areas like emergency response. Stakeholders need stable, certifiable AI behavior to build trust [10]. However, natural language coordination can be variable and ambiguous. Balancing this may involve validated prompt templates or incorporating human approval for critical tasks. Stakeholder feedback can refine prompt structures, set fallback defaults and establish evaluation criteria.

6 Conclusion

This work examines LLMs' ability to generate runnable code for sensor tasks like motion detection and QR-code authentication, evaluating their performance in creating accurate pipelines. Claude and GPT-4o showed promise but struggled with robustness, error correction, and new libraries. While lightweight models allow on-device use, real-world deployment needs better auditability, adaptability, and inclusivity. Future research should guide LLMs into broader frameworks, integrating human and system feedback.

Acknowledgment

This work was supported by the European Union and Estonian Research Council via project TEM-TA101. Thanks to our supervisor Huber Flores for his support.

References

- [1] H. Chen, T. Finin, and A. Joshi. 2003. An ontology for context-aware pervasive computing environments. *The knowledge engineering review* 18, 3 (2003), 197–207.
- [2] B. Greenstein and B. Longstaff. 2011. Followme: Enhancing mobile applications with open infrastructure sensing. In *Proceedings of ACM HotMobile '11*, 105–110.
- [3] S. Jain, D. Calacci, and A. Wilson. 2024. As an AI Language Model, I Would Recommend Calling the Police: Norm Inconsistency in LLM Decision-Making. In *Proceedings of AIES '24*, Vol. 7, 624–633.
- [4] R. Kuchida et al. 2025. Assembling Edge Intelligence and Decentralized Infrastructure on Command using LLMs. In *IEEE INFOCOM WKSHPS 2025*.
- [5] C. Mathias et al. 2021. *Opportunity++: A Multimodal Dataset for Video- and Wearable, Object and Ambient Sensors-based Human Activity Recognition*.
- [6] S. Pal, A. Dorri, and R. Jurdak. 2022. Blockchain for IoT access control: Recent trends and future research directions. *Journal of Network and Computer Applications* 203 (2022), 103371.
- [7] K. Post et al. 2025. ContextLLM: Meaningful Context Reasoning from Multi-Sensor and Multi-Device Data Using LLMs. In *Proceedings of ACM HotMobile'25*.
- [8] W. J. Powers et al. 2019. Guidelines for the early management of patients with acute ischemic stroke: 2019 update to the 2018 guidelines for the early management of acute ischemic stroke: a guideline for healthcare professionals from the American Heart Association/American Stroke Association. *Stroke* (2019).
- [9] M. Silva et al. 2020. Visually-defined real-time orchestration of iot systems. In *MobiQuitous '20*, 225–235.
- [10] S. Tirupathi et al. 2025. GAF-Guard: An Agentic Framework for Risk Management and Governance in Large Language Models.
- [11] S. H. Vemprala et al. 2024. ChatGPT for Robotics: Design Principles and Model Abilities. *IEEE Access* (2024).
- [12] M. Yamagami et al. 2023. How do people with limited movement personalize upper-body gestures? considerations for the design of personalized and accessible gesture interfaces. In *ACM ASSETS '23*, 1–15.
- [13] Y. Yang et al. 2025. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. *arXiv preprint arXiv:2504.00587* (2025).
- [14] Q. Zhang et al. 2019. Edge video analytics for public safety: A review. *Proc. IEEE* 107, 8 (2019), 1675–1696.